UNITÀ 6 · INTRODUZIONE AI SISTEMI OPERATIVI

La gestione della CPU: i processi

Un programma è un file fermo sul disco. Quando lo avvii diventa un processo: nasce, aspetta, lavora, si ferma, muore. Oggi seguiamo la sua vita, comando per comando.

3E TPSI tasklist, ps, top, stati, PCB, thread

PER COMINCIARE

Secondo te, quanti processi sono attivi adesso sul PC del laboratorio?

SONDAGGIO

- **A** Meno di 20
- **B** Tra 20 e 100
- **C** Tra 100 e 300
- **D** Più di 300

PROGRAMMA E PROCESSO

La ricetta e la torta che sta cuocendo

Ricetta

Il **programma**: istruzioni scritte in un file, ferme sul disco.

Cucinare

Il **processo**: il programma in esecuzione, con il suo stato.

Cuoco

La **CPU**: esegue un passo della ricetta alla volta.

Ingredienti

I **dati**: in memoria, letti e trasformati.

> Un programma è **passivo**, un processo è **attivo**: ha un punto a cui è arrivato, dei dati, dei file aperti. Con una ricetta si possono cuocere tre torte insieme.

METTITI ALLA PROVA

Apri tre finestre del Prompt dei comandi. Quanti programmi e quanti processi?

QUIZ

A 3 programmi e 3 processi

B 1 programma e 1 processo con tre finestre

C 1 programma e 3 processi

D 3 programmi e 1 processo

AL COMPUTER · 15 minuti

I processi di Windows con tasklist

1. Apri tre finestre del **Prompt dei comandi** (cerca "cmd" nel menu Start).
2. In una scrivi `tasklist /fi "imagename eq cmd.exe"` e annota i PID.
3. Scrivi `tasklist > processi.txt` e poi `notepad processi.txt`: conta le righe.
4. Chiudi una finestra con `taskkill /pid N`, dove N è il PID di un'**altra** finestra.
5. Ripeti il punto 2: quanti cmd.exe restano?

Controllo: al punto 2 compaiono 3 righe cmd.exe con 3 PID diversi; dopo taskkill ne restano 2. In processi.txt ci sono centinaia di processi, tra cui System con PID 4.

MENTI CHE CI HANNO PRECEDUTO

Gli inventori di Unix e il processo

> **Un'immagine è lo stato attuale di uno pseudo-computer. Un processo è l'esecuzione di un'immagine.**

Dennis Ritchie e Ken Thompson, "The UNIX Time-Sharing System", 1974

Che cosa c'è nell'immagine

La memoria, i registri della CPU, i file aperti, la cartella corrente. Il sistema operativo la conserva in una struttura: il PCB.

Unix, 1969

Nato ai Bell Labs su un minicomputer PDP-7. Le sue idee vivono in Linux, Android e macOS: il comando `ps` esiste dagli anni Settanta.

PROCESS CONTROL BLOCK

La carta d'identità di ogni processo

Campo del PCB	Esempio
PID, identificatore	4812
Stato	pronto
Program counter	prossima istruzione da eseguire
Registri della CPU	valori salvati all'ultima pausa
Memoria assegnata	dove sono codice e dati
File aperti e dispositivi	processi.txt
Priorità e tempo di CPU usato	normale, 0,8 s

Perché serve

La CPU è una sola (o poche), i processi sono centinaia. Quando un processo perde la CPU, il PCB conserva **il punto esatto** in cui era arrivato, come un segnalibro.

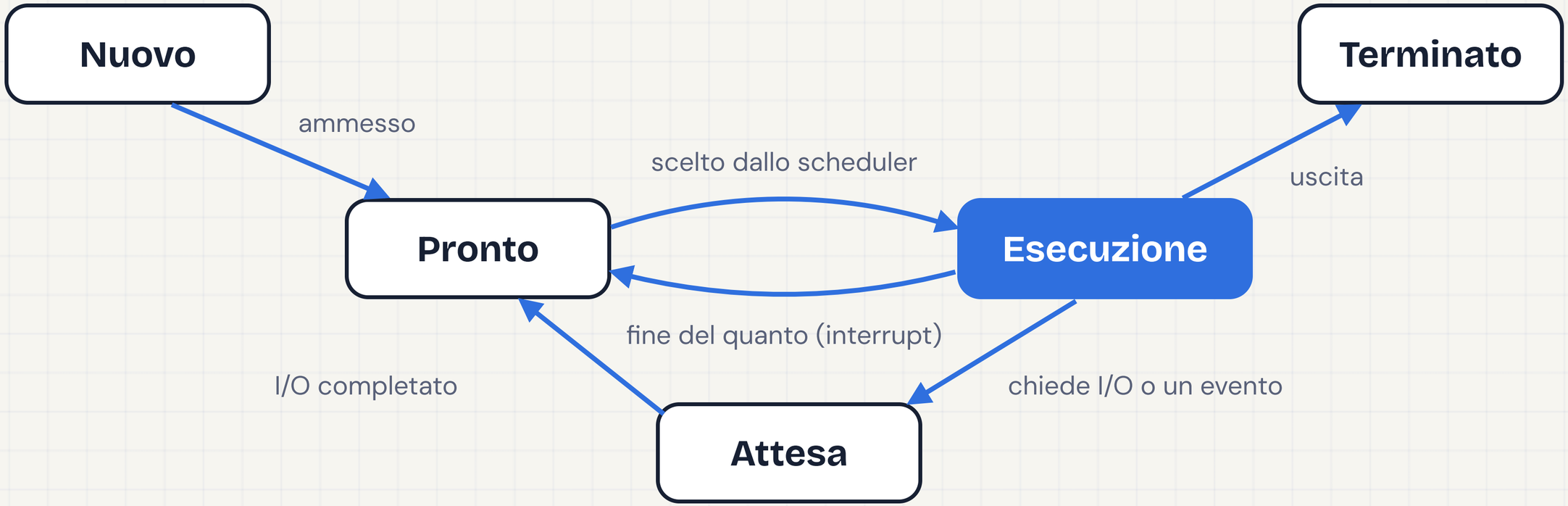
PUZZLE

Abbina ogni campo del PCB alla domanda a cui risponde

PID	---	Quali documenti e dispositivi stai usando?
Stato	---	Quanto sei importante rispetto agli altri?
Program counter	---	Qual è la prossima istruzione da eseguire?
Registri	---	Stai lavorando, aspetti la CPU o aspetti un dato?
File aperti	---	Che valori avevi nella CPU quando ti hanno fermato?
Priorità	---	Chi sei, fra tutti i processi?

GLI STATI DI UN PROCESSO

Il modello a cinque stati



QUIZ A TRABOCCHETTO

Un processo aspettava un file dal disco. Il file arriva. In che stato passa?

QUIZ

A In esecuzione, subito

B Pronto

C Resta in attesa finché l'utente clicca

D Terminato

PUZZLE

La vita di Calc in un'ora di lezione: metti in ordine

1. I dati arrivano: Calc torna in coda (pronto)
2. Riprende la CPU e ricalcola le formule (esecuzione)
3. Chiudi Calc: memoria e PCB vengono liberati (terminato)
4. Il programma è in memoria e aspetta la CPU (pronto)
5. Apri un file: Calc aspetta i dati dal disco (attesa)
6. Fai doppio clic su Calc: il sistema crea il PCB (nuovo)
7. Lo scheduler gli assegna la CPU (esecuzione)

AL COMPUTER · 20 minuti

Processi Linux sotto la lente, in JSLinux

1. Apri **bellard.org/jslinux** e avvia una console Linux. Scrivi `ps`: chi ha il PID 1?
2. Scrivi `sleep 300 &` e poi `grep State /proc/$!/status`
3. Scrivi `yes > /dev/null &`, ripeti il `grep`, poi `top` (esci con q).
4. Scrivi `kill -STOP $!`, ripeti il `grep`; poi `kill $!`.
5. Annota i tre stati in **stati-linux.txt**.

Controllo: sleep è **S** (sleeping, in attesa), yes è **R** (running, pronto o in esecuzione) e in top usa quasi tutta la CPU, dopo STOP è **T** (stopped). Il PID 1 è **init**, il primo processo.

IL CAMBIO DI CONTESTO

Passare la CPU da un processo all'altro

1

Ferma A

Un interrupt o una richiesta di I/O interrompe il processo A.

2

Salva

Il kernel copia registri e program counter nel PCB di A.

3

Carica

Lo scheduler sceglie B e ricarica i valori dal PCB di B.

4

Riparte B

B continua esattamente da dove si era fermato.

Durante il cambio di contesto la CPU **non fa lavoro utile** per nessuno: costa qualche microsecondo. Farne troppi peggiora le prestazioni.

PROCESSI E THREAD

Un processo può avere più "fili" di esecuzione

Processo

Ha la **sua memoria**, protetta dagli altri. Se si blocca, gli altri processi continuano.

Thread

Vive dentro un processo e ne **condivide la memoria**. Ha solo il suo program counter, i suoi registri e il suo stack.



AL COMPUTER · 15 minuti

Quanti processi ha il tuo browser?

1. Apri il browser con **una sola scheda**. In cmd scrivi `tasklist /fi "imagename eq chrome.exe"` (per Edge: `msedge.exe`) e conta le righe.
2. Apri 5 schede su 5 siti diversi e ripeti il conteggio.
3. In Gestione attività, scheda **Dettagli**, fai clic destro sull'intestazione, **Seleziona colonne**, e aggiungi **Thread**.
4. Annota in **browser-processi.txt**: processi con 1 scheda, con 6 schede, thread del processo più grande.

Controllo: già con una scheda i processi sono più di uno (GPU, rete, estensioni…). Con siti diversi il numero cresce; ogni processo ha più thread, spesso decine.

OGGI

Chi decide quali processi sopravvivono?





SFIDA FINALE

Apri il lucchetto con quello che hai imparato



- 1ª cifra: quanti stati ha il modello visto oggi
- 2ª cifra: il PID del primo processo di Linux, init
- 3ª cifra: quanti processi crei aprendo tre finestre del Prompt dei comandi
- 4ª cifra: quante frecce escono dallo stato Esecuzione

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Un programma in esecuzione si chiama _____.

Il numero che identifica un processo si chiama _____.

Un processo che aspetta un dato dal disco è nello stato di _____.

Salvare il PCB di un processo e caricare quello di un altro si chiama cambio di _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Programma = file fermo; processo = programma in esecuzione
- Stati: nuovo, pronto, esecuzione, attesa, terminato
- Il PCB conserva PID, stato, registri, memoria, file
- Cambio di contesto: salva un PCB, carica l'altro
- I thread condividono la memoria del processo
- tasklist e taskkill; ps, top e /proc in Linux

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.